

Fast Computation of Multivariate Subresultants

Yunus GÜRLEK, Frederic XIA

Supervisor: Weija WANG

May 15, 2026

Outline

- 1 Introduction
- 2 Resultants
- 3 Subresultants
- 4 Multivariate Polynomials
- 5 Proposed Algorithm
- 6 Benchmarks
- 7 Conclusion

Goal

Problem

Given $f, g \in R[x]$ with $\deg(f) = p > q = \deg(g)$, compute

$$\gcd(f, g), \quad \text{Res}(f, g), \quad (\text{sResP}_i(f, g))_{0 \leq i \leq q}.$$

Main idea

Use subresultants and specialization to replace expensive symbolic computations by many cheaper univariate computations.

Euclidean Algorithm: Correct but Costly

$$r_0 = f, \quad r_1 = g, \quad r_{i+1} = r_{i-1} \bmod r_i.$$

The last non-zero remainder is $\gcd(f, g)$.

Issue

Over rings such as $\mathbb{Z}$, coefficient sizes may grow quickly during division.

Example: Coefficient Growth

$$f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5,$$

$$g = 3x^6 + 5x^4 - 4x^2 - 9x + 21.$$

$$r_2 = -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, \quad r_3 = -\frac{117}{25}x^2 - 9x + \frac{441}{25},$$

$$r_4 = \frac{233150}{19773}x - \frac{102500}{6591}, \quad r_5 = -\frac{1288744821}{543589225}.$$

$$\gcd(f, g) = 1.$$

Key property

An appropriate subsequence of

$$(\text{sResP}_i(f, g))_{0 \leq i \leq q}$$

is proportional to the Euclidean remainder sequence.

They are defined by determinants, so coefficients remain in the base ring.

Example: Subresultants

For the same f and g :

$$\begin{aligned} \text{sResP}_5(f, g) &= 15x^4 - 3x^2 + 9, & \text{sResP}_3(f, g) &= 65x^2 + 125x - 245, \\ \text{sResP}_2(f, g) &= 9326x - 12300, & \text{sResP}_0(f, g) &= 260708. \end{aligned}$$

$$\gcd(f, g) = 1.$$

- Subresultants were already known to Euler.
- Collins introduced their use in computer algebra.
- Lazard and Ducos improved the subresultant algorithm.
- FLINT and Maple use subresultant-based methods for resultants.
- Wolfram exposes `SubresultantPolynomials` directly.

Contribution

This work

Design a fast algorithm for subresultants over

$$\mathbb{Q}, \quad \mathbb{Z}, \quad \mathbb{F}_p.$$

Technique

Use specialization and interpolation, generalizing Collins's resultant algorithm.

Observed performance

Speedups up to $20\times$ on benchmarks with total degree 30 and coefficient bit size 100.

Sylvester Matrix

Let

$$f = a_p x^p + \cdots + a_0, \quad g = b_q x^q + \cdots + b_0.$$

The Sylvester matrix represents

$$\phi : R[x]_{<q} \oplus R[x]_{<p} \rightarrow R[x]_{<p+q}, \quad (u, v) \mapsto uf + vg.$$

Resultant

$$\text{Res}(f, g) = \det S(f, g).$$

Resultant and GCD

Assume R is a domain with fraction field $\mathbb{K}$.

Criterion

$$\text{Res}(f, g) \neq 0 \iff \gcd(f, g) = 1 \text{ in } \mathbb{K}[x].$$

No proof included.

Subresultant Polynomial

For $0 \leq i \leq q$ and $0 \leq j \leq i$, let $M_{i,j}(f, g)$ be the corresponding Sylvester-Habicht minor.

Definition

$$\text{sResP}_i(f, g) = \sum_{j=0}^i \det(M_{i,j}(f, g)) x^j.$$

Non-defective case

$$\deg(\text{sResP}_i(f, g)) = i.$$

Structure Theorem: Operational Meaning

- Non-zero subresultants encode Euclidean remainders.
- Defective subresultants reveal degree drops.
- Zero subresultants indicate gaps in the sequence.

Consequence

The GCD and resultant can be computed without division in the fraction field.

Subresultant Algorithm: Skeleton

- 1 Initialize

$$\text{sResP}_p = f, \quad \text{sResP}_{p-1} = g.$$

- 2 Scale the first subresultant:

$$\text{sResP}_q = (-1)^{(p-q)(p-q-1)/2} \text{lc}(g)^{p-q-1} g.$$

- 3 Repeatedly compute pseudo-remainders:

$$\text{sResP}_{k-1} = -\frac{\text{Rem}(s_k t_{j-1} \text{sResP}_{i-1}, \text{sResP}_{j-1})}{s_j t_{i-1}}.$$

- 4 Fill defective entries when needed.

Multivariate Setting

Let

$$f, g \in \mathbb{Z}[x_1, \dots, x_n].$$

View them as univariate polynomials in x_1 :

$$f, g \in R[x_1], \quad R = \mathbb{Z}[x_2, \dots, x_n].$$

Goal

Compute

$$\text{Res}_{x_1}(f, g), \quad \text{sResP}_i(f, g).$$

Why Specialize?

Symbolic computation over

$$R = \mathbb{Z}[x_2, \dots, x_n]$$

can be expensive.

Instead, choose evaluation points

$$\eta \in \mathbb{Z}^{n-1}$$

and compute in

$$\mathbb{Z}[x_1].$$

Workflow

evaluate $\rightarrow$ univariate computation $\rightarrow$ interpolate.

Specialization of the Resultant

Let

$$h = \text{lc}_{x_1}(f) \text{lc}_{x_1}(g).$$

Valid evaluation point

If $h(\eta) \neq 0$, then

$$\text{Res}_{x_1}(f(\eta), g(\eta)) = \text{Res}_{x_1}(f, g)(\eta).$$

Bad points

Degree drops after evaluation can break the equality.

Degree Bound

Let

$$D = \deg_{x_1}(f) \deg_{x_2, \dots, x_n}(g) + \deg_{x_1}(g) \deg_{x_2, \dots, x_n}(f).$$

Bound

$$\deg(\operatorname{Res}_{x_1}(f, g)) \leq D.$$

Interpolation cost

For $n = 2$, at most $D + 1$ evaluation points are needed.

Specialization of Subresultants

Assume

$$\deg_{x_1}(f) > \deg_{x_1}(g)$$

and $h(\eta) \neq 0$.

Property

$$\text{sResP}_i(f(\eta), g(\eta)) = \text{sResP}_i(f, g)(\eta) \quad 0 \leq i \leq \deg_{x_1}(g).$$

Each coefficient can be reconstructed by interpolation.

Non-defective Subresultants

Computing only non-defective subresultants can be faster.

But evaluation must preserve the degree pattern:

$$\deg(\text{sResP}_i(f(\eta), g(\eta))) = \deg(\text{sResP}_i(f, g)).$$

Extra condition

Avoid points where a non-defective subresultant becomes defective.

Proposed Bivariate Resultant Algorithm

For $f, g \in \mathbb{Z}[x_1, x_2]$:

$$N = \deg_{x_1}(f) \deg_{x_2}(g) + \deg_{x_1}(g) \deg_{x_2}(f).$$

① Choose $N + 1$ good evaluation points.

② Compute

$$y_i = \text{Res}(f(x_i), g(x_i)).$$

③ Interpolate

$$R = \text{interpolate}(x_i, y_i, N).$$

Choosing Evaluation Points

Implemented strategies:

- Random Points
- Random Positive Points
- Small Random Points
- Small Random Positive Points
- Ordered Points
- Ordered Positive Points

Small and ordered points usually reduce interpolation cost.

Benchmark Setup

- Apple MacBook, 8-core Apple M2 SoC
- 16 GB unified memory
- Average of 5 independent trials
- Coefficient bit size fixed at 50
- Comparison against FLINT bivariate resultant computation

FMPZ Benchmarks: Bitsize 3

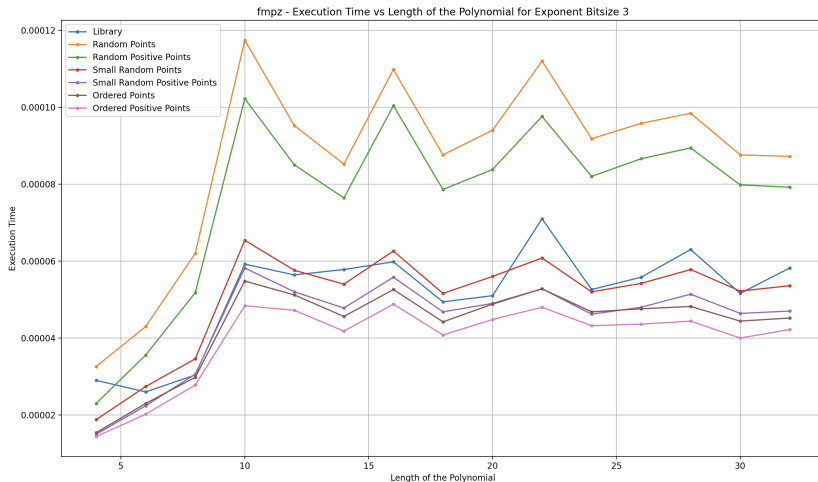


Figure: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 3.

FMPZ Benchmarks: Bitsize 4

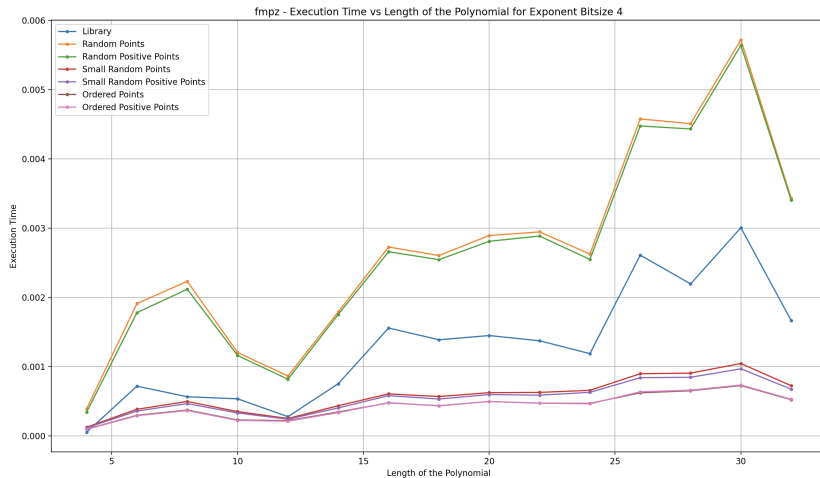


Figure: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 4.

FMPZ Benchmarks: Bitsize 5

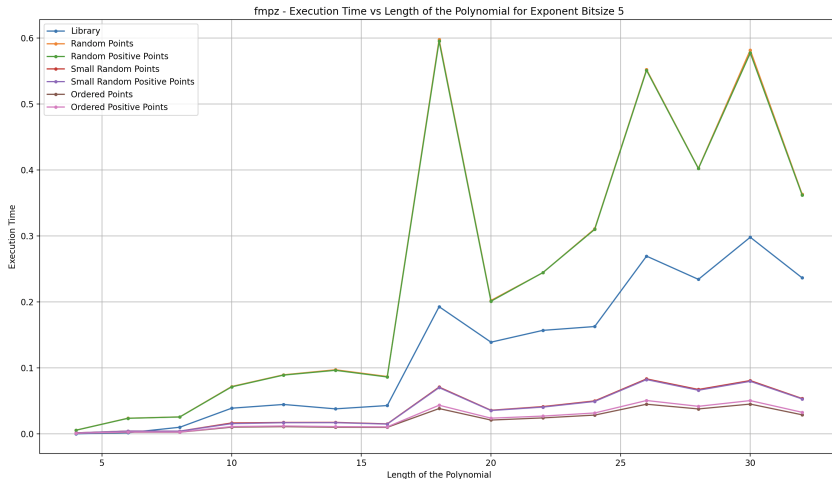


Figure: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 5.

FMPZ Benchmarks: Bitsize 6

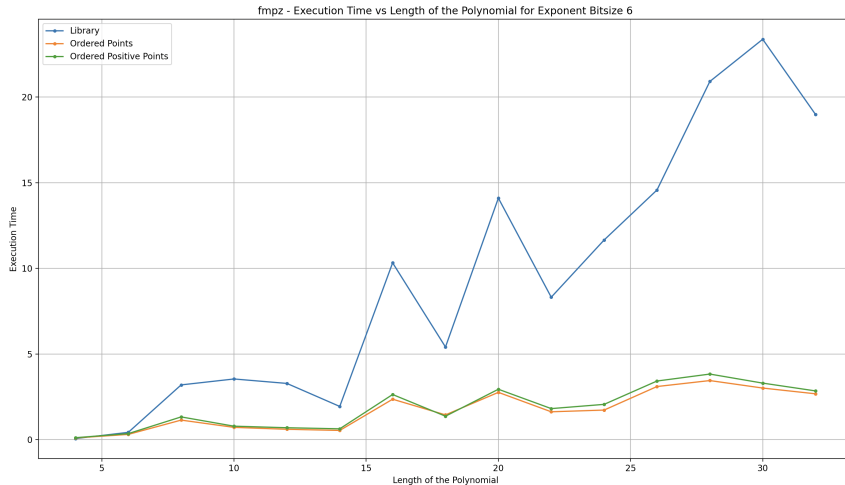


Figure: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 6.

FMPZ Benchmark Takeaways

- Fully random interpolation points are much slower.
- Point bit size strongly affects interpolation time.
- For larger exponent bitsizes, the proposed methods outperform FLINT.
- Ordered and small-random points perform similarly.

FMPQ Benchmarks: Bitsize 3

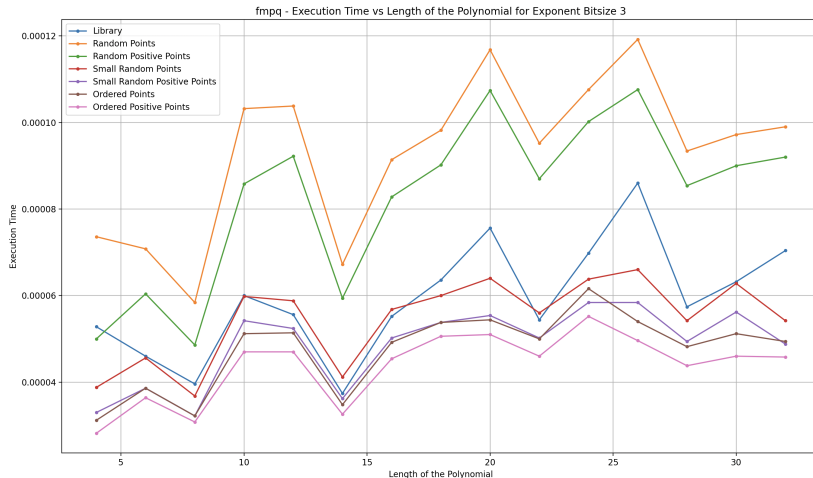


Figure: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 3.

FMPQ Benchmarks: Bitsize 4

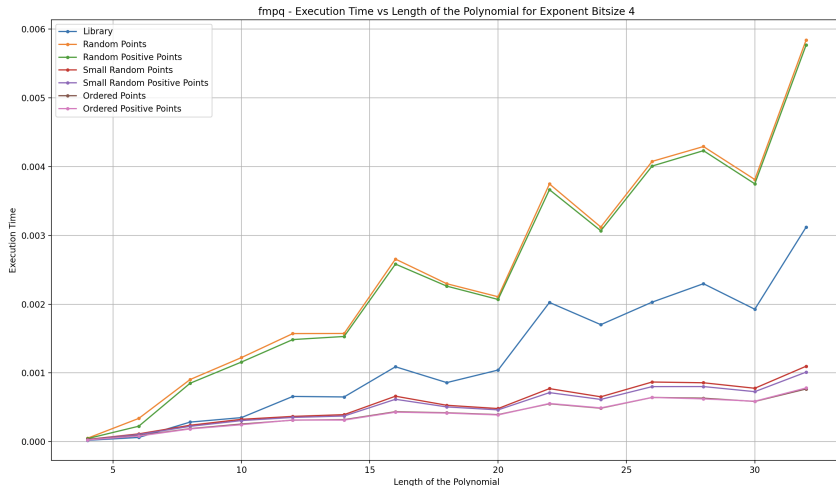


Figure: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 4.

FMPQ Benchmarks: Bitsize 5

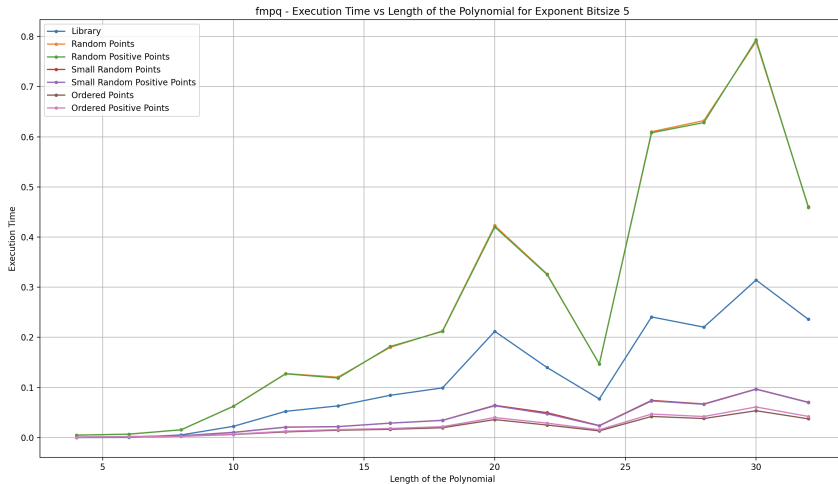


Figure: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 5.

FMPQ Benchmarks: Bitsize 6

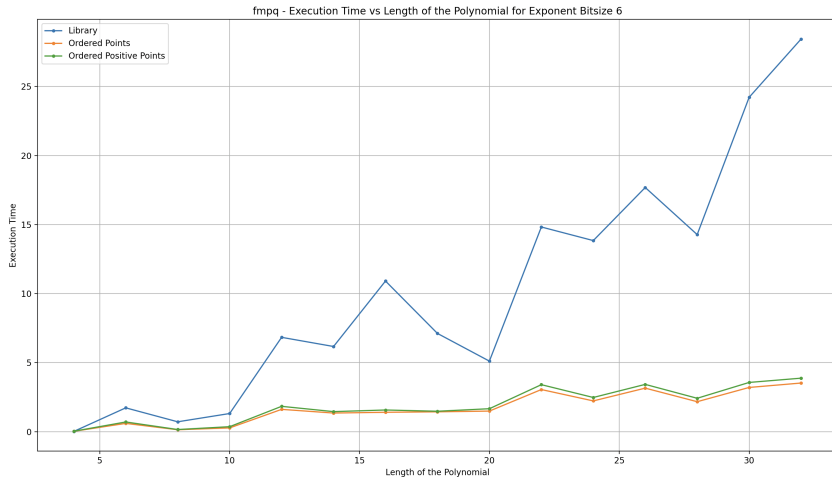


Figure: FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bitsize 6.

FMPQ Benchmark Takeaways

- Behavior is similar to the $\mathbb{Z}$ case.
- Large random points remain expensive.
- Ordered points are slightly faster in many cases.
- The gap with FLINT grows with degree and length.

- Subresultants control coefficient growth in GCD/resultant computations.
- Specialization is valid away from leading-coefficient zeros.
- Interpolation recovers multivariate resultants and subresultants.
- Evaluation point choice is critical for performance.
- Ordered or small-bit-size points work best in the benchmarks.

Next Steps

- Extend from resultants to full subresultant sequences.
- Improve handling of defective subresultants.
- Tune point selection for $\mathbb{Z}$, $\mathbb{Q}$, and $\mathbb{F}_p$.
- Compare directly against Wolfram, Maple, and FLINT interfaces.